

CS 422: Assignment 1 — Network Latencies, Ping & Traceroute

AI Use Acknowledgment: OpenAI Codex and Anthropic Claude were used to generate complete sections of code, stress-test the implementation, and draft and polish this report. All AI-generated work was reviewed and verified by the group.

Repository: <https://github.com/ColinHermack/CS422-Assignment1>

Group Members

Name	Username
Thanh Bui Minh	tbuiminh
Aaruni Gupta	gupt1066
Rishiraj Behki	rbehki
Pranith Ravella	pravella
Colin Hermack	chermack

Measurement Setup

Measurements were taken from the Purdue campus network in **West Lafayette, Indiana, USA** (public IP 128.211.248.45; 40.4259 N, -86.9081 W). [servers.csv](#) contains all **190** entries from <https://iperf3serverlist.net/>.

Both experiments and all three plots run with one command:

```
python run_all.py --servers servers.csv
```

[run_all.py](#) invokes both experiments and writes CSV data plus PDF and PNG plots. Its [build_parser\(\)](#) also exposes each experiment's options. Pass `--seed N` to repeat a Part 2 selection; without it, the script chooses a new random sample. Setup and all options are documented in [README.md](#).

Module	Role	Outputs
run_all.py	One-shot driver	all outputs below

Module	Role	Outputs
<code>part1_ping_rtt.py</code>	Part 1: ping, distance vs RTT	<code>part1_results.csv</code> , <code>plots/part1_distance_vs_rtt.{pdf,png}</code>
<code>part2_traceroute.py</code>	Part 2: traceroute, hop breakdown	<code>part2_results.csv</code> , <code>plots/part2_hop_breakdown.{pdf,png}</code> , <code>plots/part2_hopcount_vs_rtt.{pdf,png}</code>
<code>geolocation.py</code>	Shared IP→coordinates lookup	<code>geo_cache.json</code>

`load_hosts()` accepts either a CSV with an IP/HOST column or one address per line. It ignores blank lines and comments and de-duplicates the input. Both experiments use this loader.

Part 1: Ping Test and Round-Trip Time (RTT)

1(a)(i) — Ping test: min, max, average RTT

`public_ip()` obtains the runner's public IP from an external echo service. Each destination is pinged **10 times** by `ping()`, which handles Windows and Unix output and returns minimum, average, and maximum RTT in milliseconds.

Timeouts, missing replies, command errors, and DNS failures are handled without crashing; `measure()` logs and skips unusable hosts. Probes run concurrently with 20 workers.

6 of 190 hosts were dropped as non-responsive or unresolvable:

`gw1.malabo.guineanet.net`, `lg.ams-nl.gigahost.no`, `spd-desrv.hostkey.com`, `spd-fisrv.hostkey.com`, `spd-uswb.hostkey.com`, `speed.couch.ca`. The remaining **184 servers** (plus our own IP) are in `part1_results.csv`.

1(a)(ii) — Geolocation coordinates

`Geolocator` resolves hosts and obtains public coordinates from **IP2LOCATION-LITE-DB5** or, when no local database is supplied, **ip-api.com**. Lookups are cached in `geo_cache.json`; the cache preserves the IP2Location coordinates used for this report. The full coordinate table is in `part1_results.csv`.

`haversine()` computes great-circle distance on a 6,371 km sphere. A clean clone can reuse cached destination coordinates; a new public source IP may require an ip-api lookup.

1(b) — Scatter plot: distance vs RTT

Generated by `plot_distance_vs_rtt()` with one point per destination: x is great-circle distance, y is average RTT, and the whiskers show minimum-to-maximum RTT.

Graph on GitHub: [PNG](#) · [PDF](#)

The public IP did not accept hairpin traffic, so its red-star reference uses `127.0.0.1`. The 0 km / 0 ms point is therefore a disclosed loopback measurement, not a public network path.

1(c) — How distance relates to RTT

Distance and average RTT are strongly related (**Pearson $r = 0.956$** , $n = 184$). Servers within 1,000 km averaged **27.2 ms** ($n = 19$), compared with **226.5 ms** beyond 12,000 km ($n = 28$). A least-squares fit gives:

$$\text{min RTT} \approx 14.8 \text{ ms per } 1000 \text{ km} + 15.7 \text{ ms}$$

The slope is about 1.5 times the roughly 10 ms fibre propagation floor for a 1,000 km round trip. Real fibre routes are longer than great-circle distance and include router and serialization delay. The 15.7 ms intercept represents distance-independent access, campus, peering, and endpoint costs. Routing also matters: three servers geolocated 171.9 km away returned average RTTs of 7, 9, and 14 ms.

What min and max RTT tell us

Minimum RTT is the best observed path latency and therefore best reflects propagation distance plus fixed processing and serialization costs. **Maximum RTT** contains that same baseline plus variable queueing, route changes, endpoint load, and possibly deprioritized ICMP handling. Thus minimum RTT is more useful for judging distance, while the min-max spread indicates short-term network variability.

The median spread was **2.0 ms**, and **82%** of servers had a spread no larger than 3 ms. The largest variations show that distance and congestion are separate effects:

Host	Distance	min / avg / max (ms)	Spread
speedtest.lagoon.nc (New Caledonia)	12,963 km	218 / 236 / 311	93 ms
23.249.58.14	12,709 km	195 / 200 / 245	50 ms
23.249.54.234	1,744 km	40 / 44 / 80	40 ms

For example, the nearby 23.249.54.234 varied by 40 ms, while many intercontinental paths were stable. IP geolocation and great-circle distance are estimates, and the Windows ping measurements are rounded to whole milliseconds.

Part 2: Latency Breakdown

2(a) — Traceroute to 5 random destinations

`pick_destinations()` randomizes the server list, traces concurrently, and resamples after failures until five complete paths are found. It raises an error if the list is exhausted first.

`traceroute()` uses numeric `traceroute/tracert` output with a 20-hop budget, one-second probe timeout, and three probes per hop. If `traceroute` is unavailable, `_ping_traceroute()` uses TTL-limited pings. It handles failures as follows:

- A * hop is filtered out, while replies from the other probes at that hop are averaged.
- A trace is accepted only if the resolved destination IP returns a timed response. An incomplete trace is marked non-responsive and replaced with another random server.

Hop 9 did not respond on any selected path and was filtered, but all five destinations responded at their final hop. The full per-hop data are in `part2_results.csv`.

The five destinations selected on this run:

Destination	Advertised site	Hops	Destination RTT
138.199.57.129	Toronto, CA	19	24.7 ms
195.181.162.195	Miami, US	15	36.3 ms
nl.speed.vps1.net	Dronten, NL	16	100.0 ms
185.180.12.40	Vienna, AT	14	134.0 ms
105.235.237.2	Equatorial Guinea	16	223.3 ms

2(b) — Stacked bar chart: per-hop latency breakdown

Generated by `plot_hop_breakdown()` with one bar per destination. Each segment estimates the incremental latency attributed to a responsive hop. Since cumulative traceroute RTT can decrease, the code uses a backward running minimum before differencing; this keeps segments non-negative and makes them sum to destination RTT. These are estimates, while `part2_results.csv` retains the raw data. Color shows position along the path, and segments above 3 ms are labeled by hop number.

Graph on GitHub: [PNG](#) · [PDF](#)

2(c) — Scatter plot: hop count vs RTT

Generated by `plot_hopcount_vs_rtt()` with one point per destination: x is final hop count and y is destination RTT.

Graph on GitHub: [PNG](#) · [PDF](#)

2(d) — Observations

Hop count versus RTT

The five points show no reliable positive relationship ($r = -0.38$). The 19-hop Toronto path had the lowest RTT (24.7 ms), while the 16-hop Equatorial Guinea path took 223.3 ms. With only five random destinations, the negative coefficient should not be interpreted as a general inverse relationship. Hop count describes routing topology; RTT is more strongly affected by physical path length, as Part 1's larger sample demonstrates.

Latency is not spread across hops — one hop dominates

One estimated segment accounts for 39–88% of destination RTT:

Destination	Dominant hop	Jump	Share of total RTT
105.235.237.2 (Eq. Guinea)	11 → 185.35.141.155	+195.7 ms	88%
185.180.12.40 (Vienna)	11 → 171.75.8.159	+116.3 ms	87%
nl.speed.vps1.net (NL)	12 → 4.30.181.50	+87.3 ms	87%
195.181.162.195	12 → 68.86.85.58	+22.7 ms	62%
138.199.57.129 (Toronto)	11 → 213.248.91.100	+9.7 ms	39%

The dominant segment likely corresponds to a long-distance backbone span: one logical hop can contain thousands of kilometers of fibre. This explains how paths with similar hop counts can have very different RTTs.

Shared access-network cost

All paths share the same first-eight-hop pattern at roughly 2–3 ms: one CGNAT hop, six RFC1918 campus hops, and the first public border router. This common prefix contributes a small, distance-independent baseline; the larger fitted intercept in Part 1 also includes ISP, peering, and endpoint costs.

Cumulative RTT is not monotonic

Later hops can report lower RTTs: for example, `nl.speed.vps1.net` fell from 128.0 ms at hop 14 to 100.0 ms at the destination. Intermediate ICMP responses may be deprioritized, return paths may differ, and three-probe averages contain noise. The stacked chart's monotonic adjustment prevents these control-plane spikes from being misreported as negative link latency.

Reproducibility Notes

The one-shot command in Measurement Setup regenerates both CSV files and all PDF/PNG plots. Measurements naturally vary by time and source network; the runner's public IP and origin coordinates also change on another machine. Use `--seed` for repeatable Part 2 target selection and `--offline` only when the current public IP is already cached.